

Il caso di studio è la configurazione usata nel progetto: quarzo esterno da 8 MHz, CPU a 168 MHz, USB a 48 MHz. Ogni numero della schermata a fianco viene ricavato partendo dai vincoli del chip, non imparato a memoria.

[illegible]

1 Perché esiste questa schermata

Il microcontrollore non ha "una" frequenza: ne ha molte, e le decidi tu.

Ogni parte del chip è un circuito digitale sincrono: avanza di un passo a ogni fronte di un segnale di **clock**. La CPU esegue istruzioni, un timer incrementa il suo contatore, la USB trasmette un bit, sempre al ritmo del proprio clock. Cambiando la frequenza cambi quindi la velocità di tutto ciò che quel clock alimenta.

La schermata *Clock Configuration* è il disegno del **clock tree**: l'albero che parte dalle **sorgenti** (oscillatori, a sinistra) e arriva alle **periferiche** (a destra), passando per moltiplicatori, divisori e selettori. Ogni casella è un registro del blocco RCC (*Reset and Clock Control*): CubeMX scrive per te il codice che lo configura, nella funzione `SystemClock_Config()`.

Perché non mettere tutto al massimo e basta

Tre ragioni impediscono di scegliere i numeri a caso:

1. **Limiti fisici.** Ogni dominio ha una frequenza massima oltre la quale i circuiti non garantiscono il funzionamento: 168 MHz per la CPU, 42 MHz per il bus APB1, 84 MHz per APB2.
2. **Frequenze obbligate.** Alcune periferiche vogliono un valore preciso, non "il più possibile": la USB richiede **esattamente 48 MHz**.
3. **Consumo.** La potenza dinamica di un circuito CMOS cresce con la frequenza: un clock più alto del necessario è solo energia sprecata e calore.

Come leggere il disegno

Elemento	Significato
Casella azzurra	valore impostabile (sorgente o divisore)
Casella bianca	frequenza risultante, calcolata da CubeMX
Casella grigia	blocco non attivo (es. LSE se non abilitato in RCC)
Casella rossa	vincolo violato: valore fuori dai limiti
Trapezio con pallini	multiplexer (Mux): sceglie una sola delle sue entrate
/ n	divisore (prescaler): la frequenza viene divisa per n
X n	moltiplicatore (solo dentro il PLL)
Testo azzurro sotto una casella	limite massimo (es. "42 MHz max")

Regola generale

Si lavora sempre **da sinistra a destra**: sorgente → PLL → frequenza di sistema → bus → periferiche. Ogni frequenza a destra è il prodotto di quelle a sinistra per i fattori lungo il percorso.

2 Le quattro sorgenti

La colonna di sinistra: da dove nasce il ritmo.

Sorgente	Tipo	Frequenza	Precisione	A cosa serve
HSI RC	oscillatore RC interno al chip	16 MHz	circa $\pm 1\%$ (tarato in fabbrica a 25 °C), varia con la temperatura	clock di avvio: al reset il chip parte sempre da qui
HSE	quarzo esterno (sulla scheda)	4–26 MHz; 8 MHz sulla nostra scheda	decine di ppm	sorgente precisa per il PLL e per la USB
LSI RC	RC interno lento	≈ 32 kHz	molto imprecisa	watchdog indipendente (IWDG): deve funzionare anche se il clock principale si guasta
LSE	quarzo esterno lento	32.768 kHz	ppm	orologio RTC (grigio nello screenshot: non abilitato)

Perché la USB obbliga a usare l'HSE

La USB full-speed trasmette a 12 Mbit/s e la norma ammette uno scostamento di frequenza di $\pm 0.25\%$. L'HSI garantisce circa $\pm 1\%$: quattro volte peggio del tollerato. Un quarzo invece sbaglia di qualche decina di parti per milione, cioè circa 0.005%.

Quindi: **con la USB attiva la sorgente deve essere l'HSE**. Per questo in *Pinout* $\rightarrow$ *RCC* si imposta `HSE: Crystal/Ceramic Resonator`, e i pin `PH0/PH1` diventano `RCC_OSC_IN/OUT`, collegati al quarzo.

Da verificare sulla propria scheda

Il valore "Input frequency" (8) va scritto a mano e deve coincidere con il quarzo montato: è stampato sul suo involucro (es. `8.000`). Se è sbagliato, CubeMX calcola frequenze che il chip non avrà davvero.

Perché 32.768 kHz per l'orologio

Non è un numero strano: è una potenza di due.

$$32\,768 = 2^{15}$$

Un contatore binario a 15 bit che riceve 32 768 impulsi al secondo si azzerà esattamente una volta al secondo. Dividere per una potenza di due in hardware è banale (basta una catena di flip-flop): ecco perché tutti gli orologi al quarzo, da polso compresi, usano questa frequenza.

Perché il chip parte dall'HSI

L'HSI non ha bisogno di componenti esterni e si stabilizza in pochi microsecondi, mentre un quarzo impiega qualche millisecondo ad avviarsi e potrebbe anche non esserci. Il codice generato parte quindi dall'HSI, accende l'HSE, aspetta che sia stabile, configura il PLL e solo alla fine commuta il clock di sistema.

3 Il PLL: M, N, P, Q

L'unico blocco che può alzare la frequenza. E il cuore dei calcoli.

Un oscillatore a quarzo da 8 MHz non può dare 168 MHz. Il **PLL** (*Phase-Locked Loop*) contiene un oscillatore veloce, il **VCO**, la cui frequenza viene tenuta "agganciata" a un multiplo esatto del riferimento. Il risultato ha la precisione del quarzo e la velocità del VCO.

Il percorso ha quattro parametri:

```
HSE 8 MHz -> /M -> f_in -> xN -> f_VCO -+> /P -> SYSCLK
+> /Q -> 48 MHz (USB)
```

$$f_{in} = \frac{f_{HSE}}{M} \quad f_{VCO} = f_{in} \cdot N \quad f_{PLLCLK} = \frac{f_{VCO}}{P} \quad f_{48} = \frac{f_{VCO}}{Q}$$

Grandezza	Vincolo	Perché
f_{in}	1–2 MHz	campo di lavoro del comparatore di fase del PLL
f_{VCO}	100–432 MHz	campo in cui il VCO oscilla in modo stabile
P	solo 2, 4, 6, 8	divisore hardware con valori fissi
f_{PLLCLK}	≤ 168 MHz	massimo della CPU
f_{48}	= 48 MHz se c'è la USB	la USB vuole 4×12 MHz esatti

Calcolo: da dove viene 336

Lo stesso f_{VCO} deve dare *contemporaneamente* 168 MHz (dividendo per P) e 48 MHz (dividendo per Q), con P e Q interi:

$$f_{VCO} = 168 P = 48 Q$$

Provo i valori di P ammessi:

P	f_{VCO}	$Q = f_{VCO}/48$	Valido?
2	336 MHz	7	sì
4	672 MHz	14	no: oltre 432 MHz
6, 8	1008, 1344 MHz	–	no

Quindi l'unica possibilità è $P = 2$, $f_{VCO} = 336$ MHz, $Q = 7$. Resta da scegliere come arrivare a 336 partendo da 8 MHz, con f_{in} tra 1 e 2 MHz:

M	f_{in}	$N = 336/f_{in}$	Nota
8	1 MHz	336	quella usata nel progetto: conti immediati
4	2 MHz	168	il Reference Manual raccomanda 2 MHz per ridurre il jitter del PLL

In pratica

Frequenze identiche; $M = 4$, $N = 168$ ha un po' meno jitter, irrilevante per questo progetto.

4 Clock di sistema, AHB e bus APB

Dalla frequenza della CPU a quella di ogni periferica.

System Clock Mux → SYSCLK

Sceglie chi alimenta tutto il resto: **HSI**, **HSE** o **PLLCLK**. Per avere 168 MHz va scelto PLLCLK. Con HSI o HSE la CPU andrebbe a 16 o 8 MHz (è la situazione del primo blink, che girava a 16 MHz).

AHB Prescaler → HCLK

Il bus AHB collega CPU, memorie e DMA. Dividere qui rallenterebbe tutto, CPU compresa: si lascia /1, quindi HCLK = 168 MHz. Da HCLK escono:

- **FCLK Cortex**: il clock della CPU (168 MHz).
- **Cortex System timer**: il timer SysTick (/1 oppure /8). La HAL lo usa per contare i millisecondi di `HAL_Delay()`: con 168 MHz ricarica il contatore ogni 168 000 cicli, cioè ogni 1 ms.
- **Ethernet PTP**: clock per i timestamp di rete (non usato).

APB1 e APB2: i bus delle periferiche

Bus	Massimo	Prescaler	PCLK	Clock timer
APB1 (lento)	42 MHz	/4	168/4 = 42	84 MHz
APB2 (veloce)	84 MHz	/2	168/2 = 84	168 MHz

I prescaler non sono una scelta: sono il divisore più piccolo che rispetta il massimo. Con /2 APB1 sarebbe a 84 MHz, il doppio del consentito.

La regola del ×2 dei timer

Nello screenshot ci sono due caselle "X 2". Il Reference Manual stabilisce che:

Regola

se il prescaler di un bus APB vale 1, i timer su quel bus ricevono PCLK;
se vale più di 1, ricevono $2 \times \text{PCLK}$.

$$f_{TIM,APB1} = 2 \cdot \frac{168}{4} = 84 \text{ MHz} \quad f_{TIM,APB2} = 2 \cdot \frac{168}{2} = 168 \text{ MHz}$$

Conseguenza pratica: un timer sta su APB1 o APB2 a seconda del modello (il datasheet lo indica per ognuno). Per calcolare un PWM bisogna partire dal clock del timer, non da PCLK.

Applicazione: il PWM dell'ESC

L'ESC vuole un impulso ogni 20 ms (50 Hz), con larghezza tra 544 e 1472 μs . Con un timer su APB1 (84 MHz):

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1)(ARR + 1)}$$

1. Scelgo $PSC = 83$: il contatore avanza a $84/(83 + 1) = 1 \text{ MHz}$, cioè **1 μs per conteggio**.
2. Scelgo $ARR = 19\,999$: un periodo dura 20 000 conteggi = 20 ms, cioè 50 Hz.
3. Il registro di confronto CCR diventa direttamente la larghezza d'impulso in μs : $CCR = 1472 \rightarrow$ motore fermo.

Il "+1" c'è perché il contatore conta da 0 a ARR compreso, e il prescaler divide per PSC+1.

5 Gli altri blocchi, e cosa CubeMX fa senza dirtelo

Quello che nello screenshot si vede ma non usiamo, e quello che non si vede ma serve.

Enable CSS

Clock Security System: sorveglia l'HSE. Se il quarzo smette di oscillare, il chip commuta da solo sull'HSI e genera un'interruzione non mascherabile (NMI), così il firmware può reagire invece di bloccarsi. Per un controllore che comanda un motore è una protezione sensata (con l'HSI il PLL non è più corretto e la USB smette di funzionare, ma il codice di sicurezza può ancora fermare il motore).

PLLI2S

Un secondo PLL dedicato all'audio digitale (interfaccia I2S), che richiede frequenze legate ai campionamenti audio. Condivide l'ingresso $f_{in} = 1$ MHz: con $\times 192$ e $/2$ dà $192/2 = 96$ MHz. Non lo usiamo: valori di default.

MCO1 e MCO2

Microcontroller Clock Output: portano un clock interno su un pin (MCO1 su PA8, MCO2 su PC9), con un divisore da 1 a 5. Utili per **verificare con un oscilloscopio** che la frequenza sia davvero quella calcolata: ad esempio $SYSCLK/4 = 42$ MHz su PC9. Si attivano dal Pinout; nello screenshot sono grigi, cioè spenti.

RTC Clock Mux

Sceglie la sorgente dell'orologio: LSE, LSI o HSE diviso. Spento finché non si attiva l'RTC.

Cose che non compaiono nella schermata

Wait state della flash. La flash non riesce a fornire un'istruzione a ogni ciclo a 168 MHz: la CPU deve aspettare alcuni cicli ("wait state") per ogni lettura. Il numero dipende da frequenza e tensione; con alimentazione tra 2.7 e 3.6 V:

HCLK	≤30	≤60	≤90	≤120	≤150	≤168
Wait state	0	1	2	3	4	5

CubeMX lo imposta da solo (`FLASH_LATENCY_5` in `SystemClock_Config`). L'acceleratore **ART** (cache e prefetch delle istruzioni) nasconde gran parte di questa attesa; la HAL lo attiva in `HAL_Init()` .

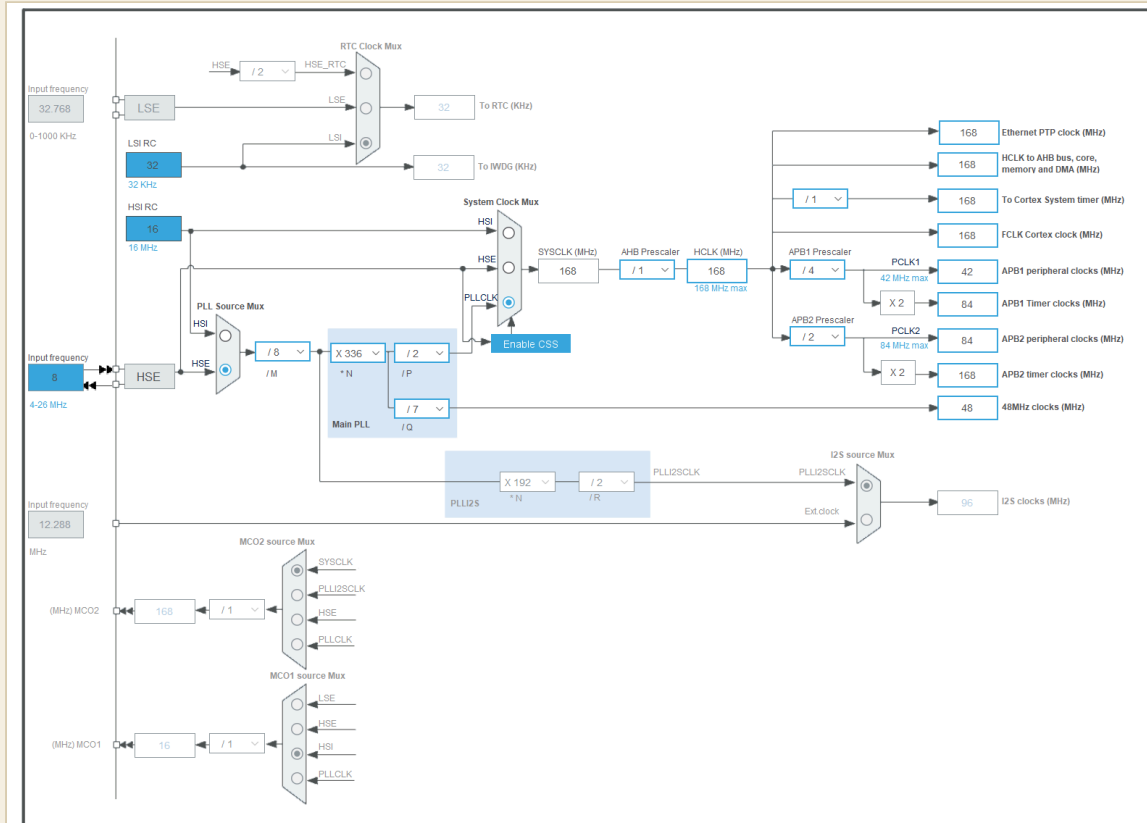
Voltage scaling. Il regolatore interno ha due modalità: Scale 1 permette 168 MHz, Scale 2 si ferma a 144 MHz ma consuma meno. CubeMX sceglie Scale 1 (`PWR_REGULATOR_VOLTAGE_SCALE1`).

Perché è importante saperlo

Se un giorno si modifica il clock a mano nel codice, alzando la frequenza senza aumentare i wait state, la CPU legge istruzioni corrotte e il programma va in crash in modi imprevedibili. L'ordine giusto è: prima i wait state, poi la frequenza.

6 La configurazione del progetto, e come leggere un errore

Tutti i numeri della schermata, ricavati in sequenza.



Il caso N = 136

Durante la configurazione, per un errore di battitura, N era rimasto **136**. CubeMX mostrava: tutto a **68 MHz** e la casella 48 MHz in **rosso** con 19.43.

Rifacendo la catena con il valore sbagliato i sintomi si spiegano tutti:

$$\frac{8}{8} \cdot 136 = 136 \Rightarrow \frac{136}{2} = 68 \text{ MHz}, \quad \frac{136}{7} = 19.43 \text{ MHz}$$

68 è la metà di 136, e 19.43 non è 48: il punto della catena dove i numeri smettono di tornare è proprio l'uscita del $\times N$.

Metodo di debug

Ripercorri la catena da sinistra a destra confrontando ogni casella con il valore atteso. Il primo valore sbagliato indica il blocco da correggere; tutto ciò che sta a destra è sbagliato solo di conseguenza.

Scorciatoia

Scrivendo **168** nella casella HCLK e premendo Invio, CubeMX cerca da solo una combinazione valida. Va comunque ricontrollato il risultato, in particolare i 48 MHz.

```
HSE 8 -/8-> 1 -x336-> VCO 336 -+-/2-> SYSCLK 168 -/1-> HCLK 168
                                     +-/-7-> USB 48
HCLK 168 -/4-> PCLK1 42 -x2-> timer APB1 84
HCLK 168 -/2-> PCLK2 84 -x2-> timer APB2 168
```

7 Procedura e checklist

Da seguire ogni volta che si configura il clock di un nuovo progetto.

Procedura

1. **Pinout** → **RCC** → **HSE**: Crystal/Ceramic Resonator .
2. **Input frequency**: il valore stampato sul quarzo (8 MHz).
3. **PLL Source Mux** su HSE.
4. Fissa le frequenze obbligate (USB 48 MHz) e quella desiderata (SYSCLK 168 MHz).
5. Trova f_{VCO} comune: $f_{VCO} = f_{SYS} \cdot P = 48 \cdot Q$, con $100 \leq f_{VCO} \leq 432$.
6. Scegli M per avere f_{in} tra 1 e 2 MHz, poi $N = f_{VCO}/f_{in}$.
7. **System Clock Mux** su PLLCLK.
8. **AHB /1; APB1 e APB2**: il divisore più piccolo che rispetta 42 e 84 MHz.
9. Annota i clock dei timer ($\times 2$) per i calcoli di PWM e interrupt periodici.
10. Nessuna casella rossa → **Generate Code**.

Checklist per la nostra scheda

Casella	Valore	Verificato da
Input frequency	8 MHz	scritta sul quarzo
M / N / P / Q	8 / 336 / 2 / 7	calcolo sezione 3
SYSCLK, HCLK	168 MHz	≤ 168 max
APB1 peripheral / timer	42 / 84 MHz	≤ 42 max
APB2 peripheral / timer	84 / 168 MHz	≤ 84 max
48MHz clocks	48 MHz	non rossa
Cortex System timer	/1 → 168 MHz	SysTick a 1 ms

Verifica sulla scheda

I numeri di CubeMX sono un calcolo: la prova è sul chip. Nel firmware di test del progetto, `HAL_RCC_GetHCLKFreq()` deve restituire 168 000 000 e la porta USB deve comparire su Windows; se il quarzo fosse diverso da 8 MHz la USB non verrebbe riconosciuta. Per una misura indipendente si può portare SYSCLK su MCO2 e guardarlo con un oscilloscopio.